

Bubble Sort vs Quick Sort

Bubble Sort

Funcionamento e lógica de ordenação: Percorre a lista várias vezes comparando elementos par a par. Se o elemento da esquerda for maior que o da direita, trocam de lugar. A cada rodada, o maior elemento restante “flutua” para sua posição correta no final.

Melhor caso: Ocorre quando a lista já se encontra totalmente ordenada no início e o algoritmo conta com uma flag (sinalizador) para interromper a execução caso nenhuma troca seja realizada na primeira rodada.

Caso médio: Ocorre quando os elementos são distribuídos de forma aleatória, exigindo múltiplos loops para comparar e trocar a maioria dos pares.

Pior caso: Ocorre quando a lista está em ordem completamente inversa em relação ao resultado desejado, forçando o número máximo de iterações e trocas.

Vantagens: É um algoritmo simples de entender e aplicar.

Limitações: Não é eficiente quando colocado contra um grande volume de dados. Isso acontece por causa do tempo de execução que cresce muito.

Uso adequado: Quantidade pequena de dados ou uma lista quase completamente ordenada.

Uso não recomendado: Qualquer aplicação real com grande volume de dados.

Quick Sort

Funcionamento e lógica de ordenação: Usa a estratégia de “dividir para conquistar”. Escolhe um elemento como pivô e particiona a lista: elementos menores que o pivô ficam à esquerda e maiores à direita. Em seguida, usando recursividade, aplica o mesmo método nas sublistas divididas.

Melhor caso: Ocorre quando o pivô escolhido em cada etapa divide a lista exatamente ao meio, criando sublistas equilibradas.

Caso médio: Ocorre na grande maioria das distribuições de dados na prática, mantendo um desempenho extremamente rápido mesmo que as divisões não sejam perfeitamente simétricas.

Pior caso: Acontece se as escolhas de pivô forem desastrosas, por exemplo, quando a lista já está ordenada e o algoritmo escolhe sistematicamente o menor ou o maior elemento como pivô.

Vantagens: Extremamente rápido na prática e com baixo consumo de memória.

Limitações: Seu pior caso pode ocorrer se a escolha do pivô não for otimizada; além disso, não é um algoritmo estável por padrão.

Uso adequado: Ordenação de grandes volumes de dados genéricos em memória RAM.

Uso não recomendado: Listas pequenas (onde a sobrecarga da recursão não compensa) ou quando é fundamental ter estabilidade garantida, sem risco de pior caso.

Tabela comparativa

Característica	Bubble Sort	Quick Sort
Princípio de funcionamento	Compara elementos par a par e troca quando estão fora de ordem; a cada rodada o maior “flutua” para o fim.	Dividir para conquistar: escolhe um pivô, particiona (menores à esquerda, maiores à direita) e aplica recursão nas sublistas.
Melhor caso	$O(n)$ — lista já ordenada, com flag que interrompe se não houver trocas.	$O(n \log n)$ — pivô divide a lista ao meio, sublistas equilibradas.
Caso médio	$O(n^2)$ — elementos aleatórios exigem múltiplos loops.	$O(n \log n)$ — muito rápido mesmo sem divisões simétricas.
Pior caso	$O(n^2)$ — lista em ordem inversa, máximo de iterações e trocas.	$O(n^2)$ — pivô mal escolhido (ex.: lista ordenada pegando sempre o menor/maior).
Uso de memória	$O(1)$ — ordena no próprio lugar (in-place).	$O(\log n)$ — in-place, mas usa a pilha de recursão.
Vantagem principal	Simples de entender e implementar.	Extremamente rápido na prática e com baixo uso de memória.
Limitação principal	Ineficiente para grande volume de dados (o tempo cresce muito).	Pior caso ruim se o pivô não for otimizado; não é estável por padrão.
Aplicação recomendada	Poucos dados ou listas quase ordenadas.	Grandes volumes de dados genéricos em RAM.